

Non-transient allocation with `std::vector` and `std::basic_string`

Document #: P3554R0 [\[Latest\]](#) [\[Status\]](#)
Date: 2025-01-05
Project: Programming Language C++
Audience: EWG
Reply-to: Barry Revzin
<barry.revzin@gmail.com>
Peter Dimov
<pdimov@gmail.com>

Contents

- [1 Introduction](#)
- [2 Proposal](#)
- [3 References](#)

§ 1 Introduction

We’ve wanted to have non-transient `constexpr` allocation for quite some time, and there have been multiple papers on this topic so far. Three dealing with the general problem [\[P0784R7\]](#) [\[P1974R0\]](#) [\[P2670R1\]](#) and one more attempting to make it work just in a narrow situation [\[P3032R2\]](#).

During discussion of the latter paper (which only talked about persisting `constexpr` variables within `constexpr` functions) in St. Louis, we took a poll to change the paper to only allow `std::vector` and `std::basic_string`, even for persistent allocation:

SF	F	N	A	SA
4	9	8	3	0

As of this writing (January 2025), we do not yet have consensus for a general design for non-transient allocation. We have ideas (see above papers), and we have a promising library workaround ([\[P3491R0\]](#)). But we know that regardless of what the general solution will end up being, `std::vector<T>` and `std::basic_string<Char, Traits, std::allocator<Char>>` will work and allow their allocations to persist. These are the most common dynamic containers. And while `define_static_array` gets us some of the way there, simply allowing these two containers to persist has a lot of ergonomic value:

With <code>define_static_array()</code>	Works By Fiat
<pre>constexpr auto ints() -> std::vector<int>; // data is a span<int const> constexpr auto data = define_static_array(ints());</pre>	<pre>constexpr auto ints() -> std::vector<int>; // data is a vector<int> constexpr auto data = ints();</pre>
<pre>constexpr auto strs() -> std::vector<std::string>; // data is a span<string_view const> constexpr auto data = define_static_array(strings() std::views::transform([&](auto&& str){ return std::string_view(define_static_string(str)); }));</pre>	<pre>constexpr auto strs() -> std::vector<std::string>; // data is a vector<string> constexpr auto data = strings();</pre>

While with a single layer of wrapping, having to add an extra call to `define_static_array` is tedious but fine, once we get into more complicated data structures, the wrapping itself gets more complicated. The good news here is that it is actually *possible* to do. The fact that the left column can exist at all after [\[P3491R0\]](#) is quite exciting.

But there are plenty of places where you might want persistent constexpr allocation — template arguments, the initializer in an expansion statement [\[P1306R2\]](#), etc. Having to perform the correct wrapping in all of these places would get old really fast. So we think that even with the addition of `std::define_static_array` and `std::define_static_string`, we’d want a little bit more help.

Additionally, this avoids the awkward issue today where whether a `std::string` is allowed to persist depends on the small string optimization strategy of the given implementation. With this proposal, it would simply always be allowed to persist.

§ 2 Proposal

The current rule is that any allocation within a constant expression E *must* be deallocated within E. That rule can be found in 7.7 [\[expr.const\]/10](#):

- (10.18) — a *new-expression*, unless either:
- (10.18.1) — the selected allocation function is a replaceable global allocation function ([`new.delete.single`], [`new.delete.array`]) and the allocated storage is deallocated within the evaluation of E, or
- (10.18.2) — [...]
- (10.19) — a *delete-expression*, unless [...]
- (10.20) — a call to an instance of `std::allocator<T>::allocate` ([`allocator.members`]), unless the allocated storage is deallocated within the evaluation of E;

We instead propose introducing the concept of eligible for persistence:

An allocation A that occurs within the evaluation of a core constant expression E is *constexpr-persistent* if:

- E is used to initialize an object O that is potentially usable in constant expressions,
- A is not deallocated within E, and
- A is *eligible for constexpr-persistence*.

And adjusting the wording as appropriate:

- (10.18) — a *new-expression*, unless either:
 - (10.18.1) — the selected allocation function is a replaceable global allocation function ([new.delete.single], [new.delete.array]) and the allocated storage is *either constexpr-persistent or* is deallocated within the evaluation of E, or
 - (10.18.2) — [...]
- (10.19) — a *delete-expression*, unless [...]
- (10.20) — a call to an instance of `std::allocator<T>::allocate` ([allocator.members]), unless the allocated storage is *either constexpr-persistent or* is deallocated within the evaluation of E;

Mark vector allocations as eligible for constexpr-persistence in 23.3.11 [vector]:

- 2 A vector meets all of the requirements of a container ([container.reqmts]), of a reversible container ([container.rev.reqmts]), of an allocator-aware container ([container.alloc.reqmts]), of a sequence container, including most of the optional sequence container requirements ([sequence.reqmts]), and, for an element type other than `bool`, of a contiguous container. The exceptions are the `push_front`, `prepend_range`, `pop_front`, and `emplace_front` member functions, which are not provided. Descriptions are provided here only for operations on vector that are not described in one of these tables or for operations where there is additional semantic information.
- x Any storage allocated for the elements of a specialization of the vector primary template is eligible for constexpr-persistence ([expr.const]).

And the same in 23.3.12 [vector.bool]:

- 1 To optimize space allocation, a partial specialization of vector for `bool` elements is provided:
- 2 Any storage allocated for the elements of the partial specialization defined in this subclause is eligible for constexpr-persistence ([expr.const]).

Mark `basic_string` allocations as eligible for constexpr-persistence in 27.4.3.1 [basic.string.general]:

- 2 A specialization of `basic_string` is a contiguous container ([`container.reqmts`]).
- x Any storage allocated for the elements of a specialization of the `basic_string` primary template is eligible for `constexpr`-persistence ([`expr.const`]).

§ 3 References

[P0784R7] Daveed Vandevoorde, Peter Dimov, Louis Dionne, Nina Ranns, Richard Smith, Daveed Vandevoorde. 2019-07-22. More `constexpr` containers.

<https://wg21.link/p0784r7>

[P1306R2] Dan Katz, Andrew Sutton, Sam Goodrick, Daveed Vandevoorde. 2024-05-07. Expansion statements.

<https://wg21.link/p1306r2>

[P1974R0] Jeff Snyder, Louis Dionne, Daveed Vandevoorde. 2020-05-15. Non-transient `constexpr` allocation using `propconst`.

<https://wg21.link/p1974r0>

[P2670R1] Barry Revzin. 2023-02-03. Non-transient `constexpr` allocation.

<https://wg21.link/p2670r1>

[P3032R2] Barry Revzin. 2024-04-16. Less transient `constexpr` allocation.

<https://wg21.link/p3032r2>

[P3491R0] Barry Revzin, Peter Dimov, Daveed Vandevoorde, Dan Katz. 2024-12-15. `define_static_{string,object,array}`.

<https://wg21.link/p3491r0>